Software Engineering · Assignment #04

Smart Campus Room Availability Checker

AI-assisted evolution, refactoring and quality evaluation

Date: 2026-11-08

AI assistant: Claude Code — model Claude Opus 5.5 (Anthropic)

Initial version: tag `v0-baseline` → tag v0-baseline Tagger: a-naimankhan <naimakhanaibar@gmail.com> Baseline: initial version for Assignment 04 9badd8c without skill
Final version: `master` @ `ff2d607`

Contents: 1. README · 2. Report (Tasks #01–#06) · 3. AI-use record · Appendices A–E: commit history, test run, mutation check, metrics, all screenshots

Smart Campus Room Availability Checker

Prototype for Software Engineering Assignment #04. Pick a room from a small fictional catalog, filter by capacity and equipment, and check whether a time interval is free. No login, payments, live APIs or database. All data is fictional and the current time is fixed at **08:00**.

Requirements

- Node.js 18+ (tested with Node 24) and npm

Setup

```
npm install
```

Run

```
npm run dev
```

Then open the URL Vite prints (default http://localhost:5173).

Test

```
npm test
```

`npm test` runs `vitest run`, which executes 49 tests in 3 files. Other checks:

```
npm run typecheck
```

```
npm run lint
```

Project structure

```
src/
  data/rooms.ts            fictional catalog (5 rooms, deep-frozen)
  domain/                  pure rules, no DOM
    time.ts                time parsing + validateInterval (AC1, AC2)
    availability.ts        overlap + checkAvailability (AC3, AC4)
    filter.ts              capacity / equipment filters (F1–F3)
    immutable.ts           deepFreeze (AC5)
    clock.ts               FIXED_NOW = 08:00
    types.ts               read-only domain types
  components/              presentation only (filters, room list, timeline, result)
  App.tsx                  UI state + wiring
tests/
  regression.test.ts       AC1–AC6 regression suite
```

```
  filter.test.ts           catalog + F1–F5
  app.ui.test.tsx          React UI integration test (jsdom)
docs/
  REPORT.md                analysis, refactoring, diagrams, quality evaluation, reflection
  evidence/                real test runs, mutation check, metrics
  screenshots/             demo scenarios
AI_LOG.md                  AI-use record
```

Versions

- Initial version: tag `v0-baseline` (commit `9badd8c`)
- Final version: `master` HEAD. Run `git log --oneline v0-baseline..HEAD` to see one commit per change.

Assignment #04 — Smart Campus Room Availability Checker

Analysis, refactoring and quality report

- **Initial version:** tag `v0-baseline` → commit `9badd8c` (the Practice #08 fixed prototype, one-file domain logic, 3 rooms).
- **Final version:** `HEAD` of branch `master` (see `git log v0-baseline..HEAD`).
- **AI assistant:** Claude Code, model Claude Opus 5.5 — full record in `AI_LOG.md`.

Task #01 — Codebase analysis

Context given to the AI assistant

1. **Repository map** (baseline):

File	Responsibility
`src/main.tsx`	React entry point
`src/App.tsx` (222 lines)	all state, time parsing, duration hint, live clock, layout of all 3 steps
`src/lib/types.ts`	`Booking`, `Room`, `CheckResult`
`src/lib/data.ts`	`ROOMS` — 3 fictional rooms (exported mutable array)
`src/lib/availability.ts`	time helpers, `intervalsOverlap`, `checkAvailability` (pure)
`src/components/RoomSelector.tsx`	room cards
`src/components/TimeField.tsx`	`<input type="time">` + unused helper `minutesToTimeInput`
`src/components/Timeline.tsx`	day timeline (bookings, candidate, "now")
`src/components/ResultPanel.tsx`	Available / Unavailable message
`src/lib/roomAvailability.test.ts`	30 Vitest unit tests

2. **Acceptance criteria to preserve:** AC1 valid time · AC2 120-minute maximum · AC3 blocked room · AC4 overlap rules (touching is allowed) · AC5 immutable booking data · AC6 clear responsive feedback.
3. **New requirements:** ≥4 rooms with id, name, capacity, equipment, blocked, bookings; F1 capacity ≥ attendees; F2 all selected equipment; F3 blocked never reported available; F4 empty state; F5 switching room refreshes the result.
4. **Constraints:** no login, payments, live APIs or database; fictional data only; fixed current time; small steps, not a rewrite.

Baseline measurements

Measure	Value	How obtained
Tests	**30 passed / 30**	`npx vitest run` → `docs/evidence/00-baseline-tests.txt`
Type check	0 errors	`npx tsc --noEmit -p tsconfig.app.json`

Measure	Value	How obtained
Lint	1 error, 1 warning	`npx eslint .` (unused import in `App.tsx`; non-component export in `TimeField.tsx`)
Rooms in catalog	3, no equipment field	`src/lib/data.ts`

Maintainability risks (with references to baseline code)

R1 — Time/interval validation is duplicated outside the domain module. The canonical rules live in `checkAvailability()` (`src/lib/availability.ts:48` start < end, `:55` inside the day, `:70` max 120 min), but they are re-implemented in the UI:

- `App.tsx:58` — `startMin === null || endMin === null` check and its error message exist **only** in the component, so this AC1 rule is not unit-testable;
- `App.tsx:82` — `endMin > startMin` to compute the duration;
- `App.tsx:86` — `duration > MAX_BOOKING_MINUTES` (copy of the AC2 rule);
- `Timeline.tsx:66` — `candidateStart >= 0 && candidateEnd <= MINUTES_PER_DAY && candidateStart < candidateEnd`. If the 120-minute limit or the day range changes, four places must change together → the UI hint and the real decision can disagree.

R2 — Booking data is a shared mutable array (AC5 risk). `src/lib/data.ts` exports `ROOMS: Room[]` without `readonly` or freezing, and `App.tsx:33` puts that same reference into state. Any `room.bookings.push(...)` or `sort()` anywhere would silently modify the "source of truth" for every later check. There was **no test** for AC5.

R3 — `App.tsx` is a god-component. It owns state, string→minute parsing, duration rules, a live clock (`setInterval` + `new Date()`, `App.tsx:20–45`) and the markup of every step; the section-heading markup is copy-pasted three times (`App.tsx:110, 128, 207`). Adding filters here would make it larger and harder to test.

R4 — The data model cannot express the new feature. `Room` (`src/lib/types.ts`) has no `equipment`; `RoomSelector` always renders every room and has no empty state, so F1–F4 have nowhere to live without touching types, data and UI at once.

R5 — Non-deterministic "now". `App.tsx` read the system clock every second, while the brief requires a fixed current time; results and screenshots depended on when the app was opened.

R6 — Dead / duplicated helpers. `TimeField.tsx:21 minutesToTimeInput()` is just `minutesToLabel()` again (and causes the lint warning); `nowClockLabel()` (`App.tsx:23–29`) re-implements the `padStart` formatting of `minutesToLabel()`; `@supabase/supabase-js` was a dependency although no database is allowed.

R7 — Test names do not match the brief. `roomAvailability.test.ts` labels its groups "AC1 — Overlap", "AC2 — Status", "AC3 — Edge cases", while the brief numbers AC1 = time, AC2 = 120 min, AC3 = blocked, AC4 = overlap, AC5 = immutability. A reviewer cannot map tests to criteria.

Duplicated-logic inventory (used later as a metric)

Found with `git grep -n` on `v0-baseline` (see Task #05):

#	Duplicate location	Canonical place
1	`App.tsx:58` null-time check + message	none in domain
2	`App.tsx:82` start < end for duration	`availability.ts:48`

#	Duplicate location	Canonical place
3	`App.tsx:86` max-duration check	`availability.ts:70`
4	`Timeline.tsx:66` interval well-formed check	`availability.ts:48,55`
5	`App.tsx:23–29` `nowClockLabel` formatting	`minutesToLabel`
6	`TimeField.tsx:21` `minutesToTimeInput`	`minutesToLabel`
7	`App.tsx:128` section heading copy	`App.tsx:110`
8	`App.tsx:207` section heading copy	`App.tsx:110`

Total: 8 duplicated logic/rendering locations.

Prioritized improvement plan (agreed before editing code)

Priority	Step	Addresses	Safety net
P0	Record baseline, tag `v0-baseline`, remove unused DB dependency, fixed current time	R5, R6	30 existing tests
P1	Align test names to AC1–AC6 and add the missing AC5 immutability test **before** refactoring	R7, R2	regression suite
P2	Split into `data/`, `domain/` (time, availability, filter), `components/`	R3	tests after each move
P3	One `validateInterval()` / `durationOf()` in `domain/time.ts`, used by domain and UI	R1	AC1/AC2 tests
P4	Deep-freeze room data, `readonly` types	R2	AC5 test
P5	Extend model with `equipment`, ≥4 rooms; add `filterRooms()`; filter UI + empty state + refresh on room switch	R4, F1–F5	new F-tests + UI test

Every step is one commit and is followed by `npx vitest run` and `npx tsc`.

AI suggestions rejected or modified

1. **Modified — shallow `Object.freeze` for AC5.** The initial plan (AI-generated, in my notes) said "bookings are not mutated (`Object.freeze` / copy)". `Object.freeze(ROOMS)` only freezes the outer array: `ROOMS[0].bookings.push(...)` would still work. I changed it to a recursive `deepFreeze()` and added a test that tries to push into a room's bookings and expects a `TypeError` (see Task #04).
2. **Modified — target structure.** The suggested layout (`main.js`, `ui/render.js`, `domain/*.js`) is for a vanilla-JS app. This project is React + TypeScript, so presentation stays in React components (`src/components/`), and only the pure parts become `src/domain/*.ts` and `src/data/rooms.ts`. A rewrite to another framework was not considered — the brief asks for small steps.
3. **Modified — "write regression tests for ~1 hour".** The plan assumed no tests existed. The baseline already had 30 passing tests that cover AC1–AC4, so instead of rewriting them I only renamed the groups to the brief's AC numbers and added the missing AC5 test.

Task #02 — Feature extension (room catalog and filters)

Catalog (`src/data/rooms.ts`): 5 fictional rooms, each with `id`, `name`, `building`, `capacity`, `equipment[]`, `blocked`, `bookings[]`.

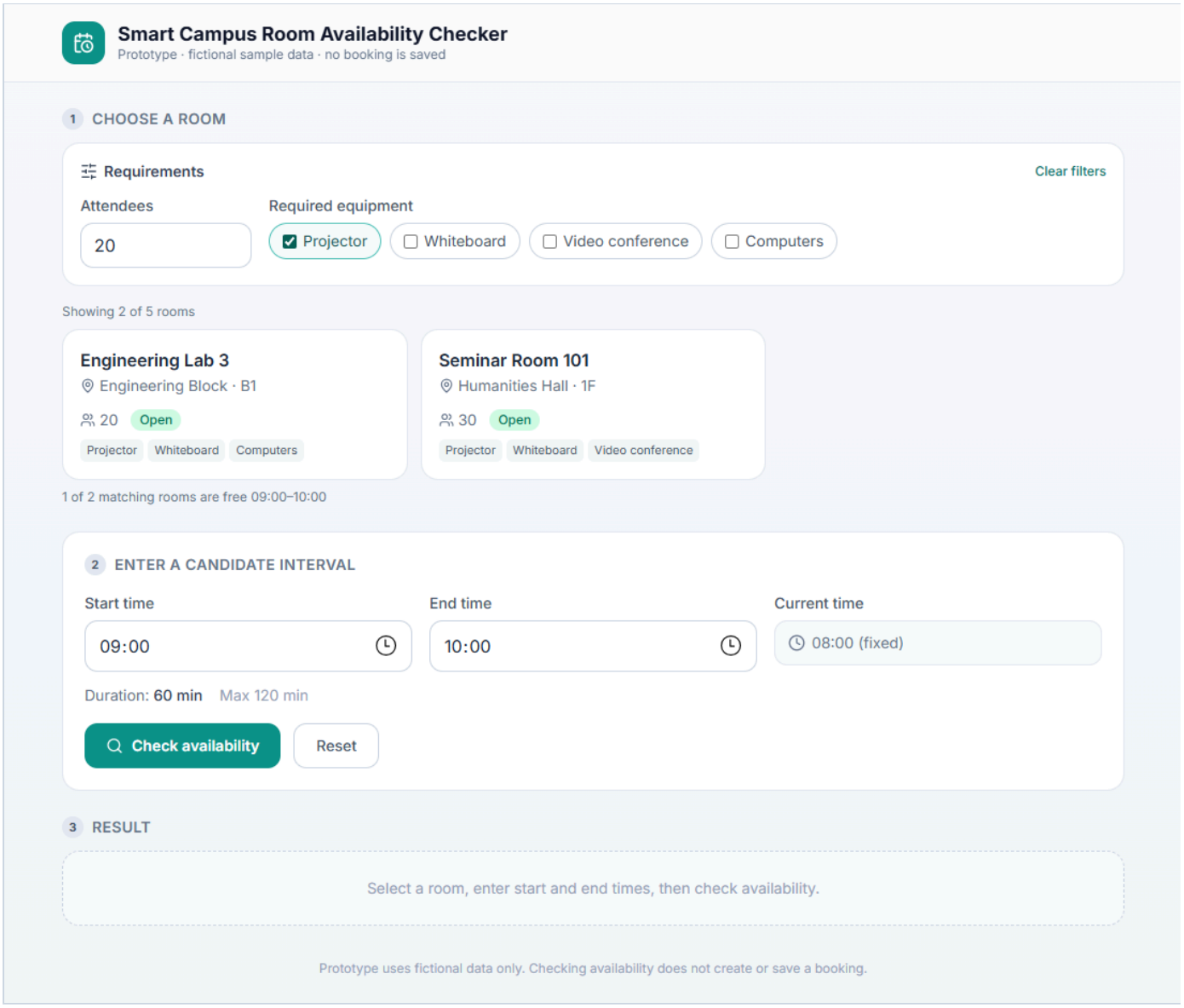
Room	Capacity	Equipment	Blocked	Bookings
Library Study Room A	8	whiteboard	no	09:00–10:00, 13:30–14:30, 19:00–20:00
Engineering Lab 3	20	projector, whiteboard, computers	no	12:00–13:00, 16:30–17:30
Design Studio 2	12	projector, whiteboard	**yes**	all day (maintenance)
Seminar Room 101	30	projector, whiteboard, video conference	no	09:00–11:00, 15:00–17:00
Quiet Pod 4	2	—	no	10:00–11:00

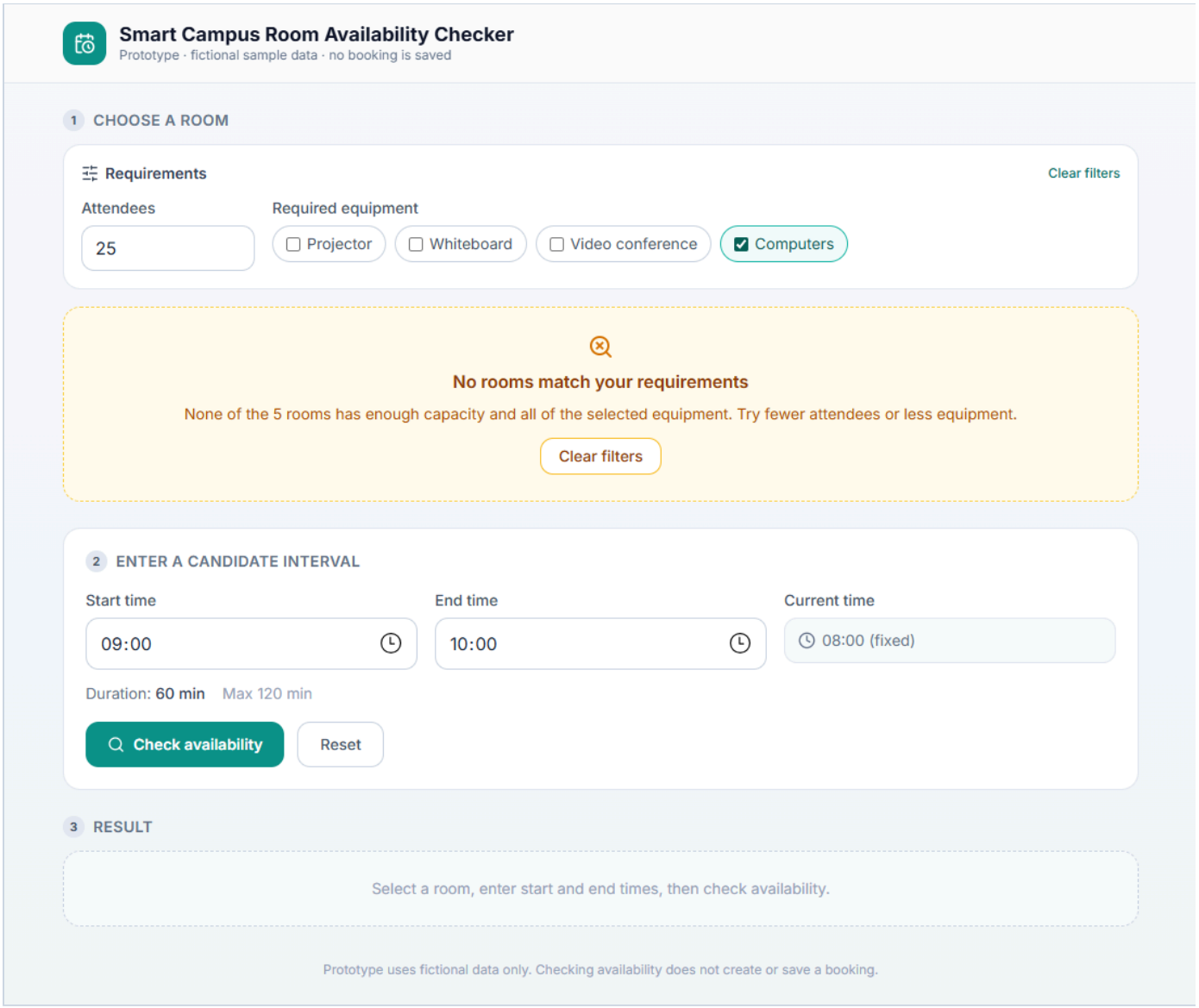
Rules (`src/domain/filter.ts`, pure functions):

- **F1** `hasCapacityFor(room, attendees)` → `room.capacity >= attendees`. An empty field means no requirement; `0`, negative, fractional or non-numeric input shows an inline error.
- **F2** `hasAllEquipment(room, required)` → `required.every(item => room.equipment.includes(item))`.
- **F3** `availableRooms(...)` re-uses `checkAvailability()` per room. A blocked room can match the filters (it is shown with a red *Blocked* badge) but is never counted or reported as available.
- **F4** `RoomSelector` renders an amber empty state ("No rooms match your requirements") with a *Clear filters* button.
- **F5** In `App.tsx` the result is *derived* (`useMemo`) from the currently selected room. After *Check availability* has been pressed, clicking another room re-runs the check against **that room's own bookings**. Changing the times clears the result as before. A room hidden by the filters is deselected.

Demonstration (screenshots in `docs/screenshots/`, captured with Playwright + Edge on the running dev server; "Observed" is the text the page showed):

#	Scenario	Observed
01	Initial page	5 of 5 rooms, fixed time 08:00
02	Successful filter: 20 attendees + Projector	"Showing 2 of 5 rooms" (Engineering Lab 3, Seminar Room 101)
03	Unsuccessful filter: 25 attendees + Computers	"No rooms match your requirements"
04	Check Library Study Room A 09:00–10:00	**Unavailable** – overlaps "Physics Study Group"
05	Switch to Engineering Lab 3 (no re-click)	**Available** – Engineering Lab 3 09:00–10:00
06	Switch to Design Studio 2	**Unavailable** – room is blocked
07	Engineering Lab 3 09:00–11:01	**Unavailable** – 121 minutes > 120
08	Engineering Lab 3 09:00–11:00	**Available** – exactly 120 minutes
09	Phone width 390 px	single column, `scrollWidth = 390` (no horizontal scroll)


Smart Campus Room Availability Checker
Prototype · fictional sample data · no booking is saved
1 CHOOSE A ROOM
Requirements
Clear filters
Attendees
20
Required equipment
Projector
Whiteboard
Video conference
Computers
Showing 2 of 5 rooms
Engineering Lab 3
Engineering Block · B1
20
Open
Projector
Whiteboard
Computers
Seminar Room 101
Humanities Hall · 1F
30
Open
Projector
Whiteboard
Video conference
1 of 2 matching rooms are free 09:00–10:00
2 ENTER A CANDIDATE INTERVAL
Start time
09:00
End time
10:00
Current time
08:00 (fixed)
Duration: 60 min
Max 120 min
Check availability
Reset
3 RESULT
Select a room, enter start and end times, then check availability.
Prototype uses fictional data only. Checking availability does not create or save a booking.


Smart Campus Room Availability Checker
Prototype · fictional sample data · no booking is saved
1 CHOOSE A ROOM
Requirements
Clear filters
Attendees
25
Required equipment
Projector
Whiteboard
Video conference
Computers
No rooms match your requirements
None of the 5 rooms has enough capacity and all of the selected equipment. Try fewer attendees or less equipment.
Clear filters
2 ENTER A CANDIDATE INTERVAL
Start time
09:00
End time
10:00
Current time
08:00 (fixed)
Duration: 60 min
Max 120 min
Check availability
Reset
3 RESULT
Select a room, enter start and end times, then check availability.
Prototype uses fictional data only. Checking availability does not create or save a booking.

Smart Campus Room Availability Checker

Prototype · fictional sample data · no booking is saved

1 CHOOSE A ROOM

Requirements Clear filters

Attendees: Any

Required equipment: ☐ Projector ☐ Whiteboard ☐ Video conference ☐ Computers

Showing 5 of 5 rooms

Library Study Room A ✓
Main Library · 2F
8 Open
Whiteboard

Engineering Lab 3
Engineering Block · B1
20 Open
Projector Whiteboard Computers

Design Studio 2
Arts Wing · 3F
12 Blocked
Projector Whiteboard

Seminar Room 101
Humanities Hall · 1F
30 Open
Projector Whiteboard Video conference

Quiet Pod 4
Student Center · 2F
2 Open
No equipment

2 of 5 matching rooms are free 09:00–10:00

2 ENTER A CANDIDATE INTERVAL

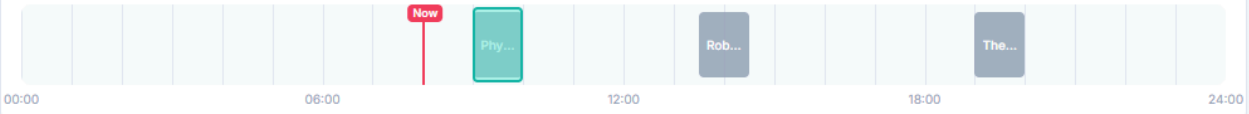
Start time: 09:00 | End time: 10:00 | Current time: 08:00 (fixed)

Duration: 60 min Max 120 min

Check availability | Reset

Today's Schedule

Booked · Your request · Now



3 RESULT

Unavailable

Overlaps an existing booking: "Physics Study Group" (09:00–10:00).

Prototype uses fictional data only. Checking availability does not create or save a booking.

Smart Campus Room Availability Checker

Prototype · fictional sample data · no booking is saved

1 CHOOSE A ROOM

Requirements Clear filters

Attendees: Any

Required equipment: ☐ Projector ☐ Whiteboard ☐ Video conference ☐ Computers

Showing 5 of 5 rooms

Library Study Room A
Main Library · 2F
8 Open
Whiteboard

Engineering Lab 3 ✓
Engineering Block · B1
20 Open
Projector Whiteboard Computers

Design Studio 2
Arts Wing · 3F
12 Blocked
Projector Whiteboard

Seminar Room 101
Humanities Hall · 1F
30 Open
Projector Whiteboard Video conference

Quiet Pod 4
Student Center · 2F
2 Open
No equipment

2 of 5 matching rooms are free 09:00–10:00

2 ENTER A CANDIDATE INTERVAL

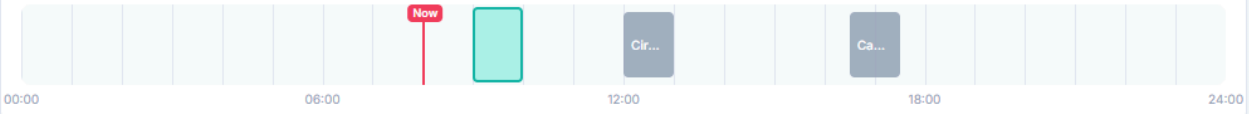
Start time: 09:00 | End time: 10:00 | Current time: 08:00 (fixed)

Duration: **60 min** Max 120 min

Check availability | Reset

Today's Schedule

Booked · Your request · Now

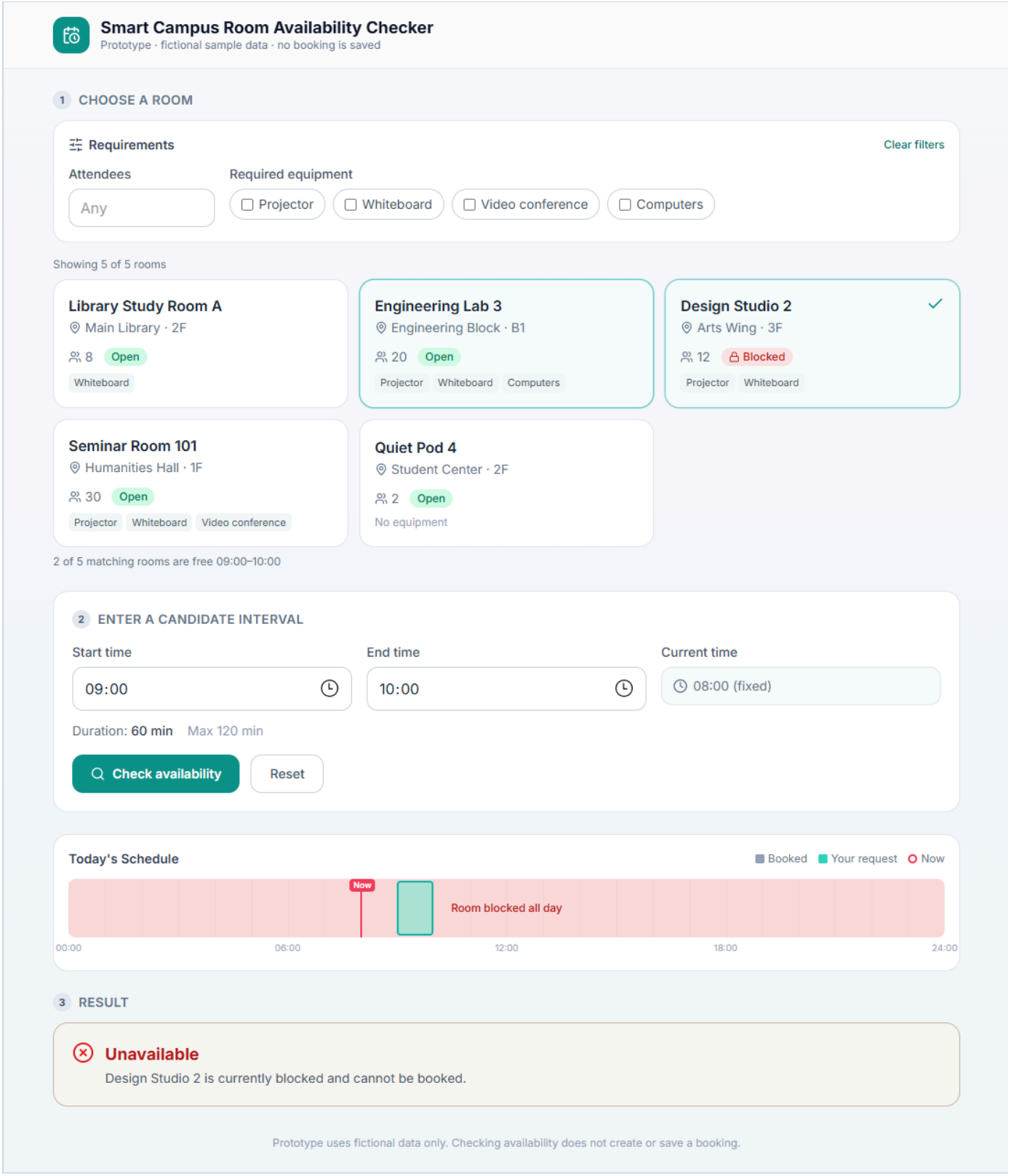


3 RESULT

Available

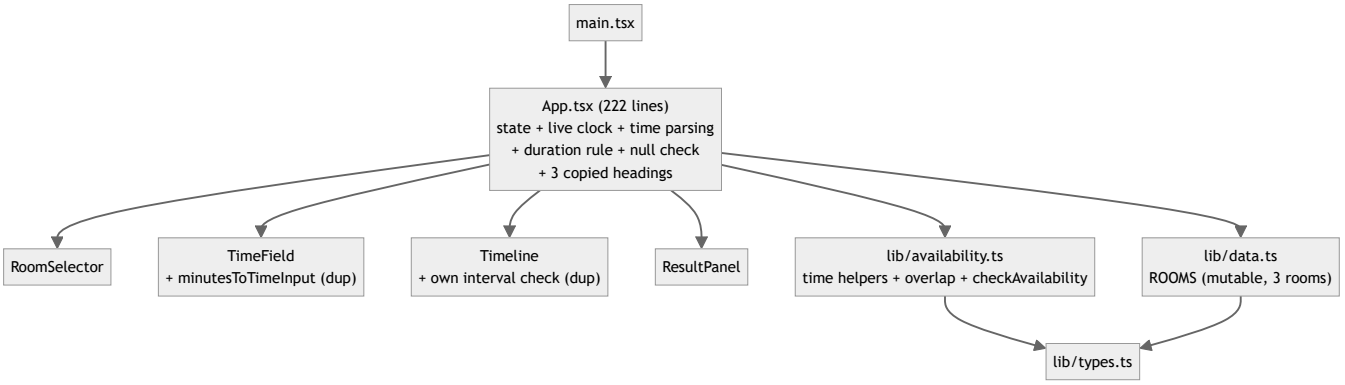
Engineering Lab 3 is available 09:00–10:00.

Prototype uses fictional data only. Checking availability does not create or save a booking.

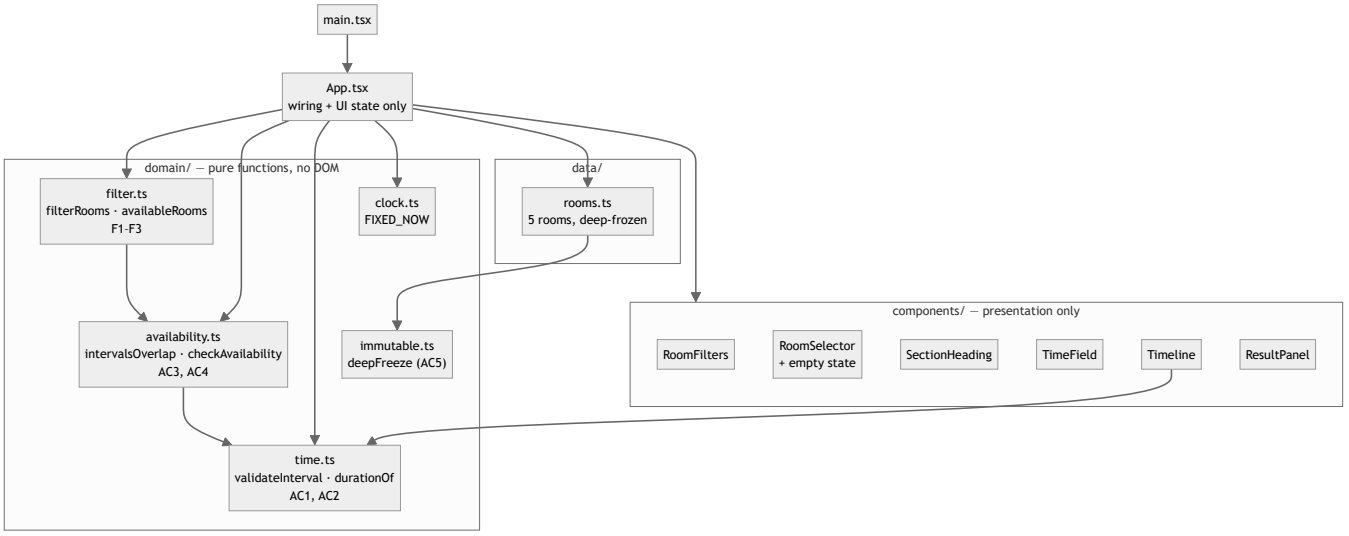


Task #03 — Refactoring

Before (`v0-baseline`)



After (`HEAD`)



Responsibilities after refactoring

Module	Responsibility	Uses DOM/React?
`src/data/rooms.ts`	fictional catalog only	no
`src/domain/time.ts`	parse/format times, **one** interval validation (AC1, AC2)	no
`src/domain/availability.ts`	overlap rule and the room decision (AC3, AC4, AC6 messages)	no
`src/domain/filter.ts`	capacity / equipment filters, available rooms (F1–F3)	no
`src/domain/immutable.ts`	`deepFreeze` (AC5)	no
`src/domain/clock.ts`	fixed current time	no
`src/domain/types.ts`	read-only `Room`, `Booking`, `Equipment`, `CheckResult`	no
`src/components/*`	rendering and user events only	yes
`src/App.tsx`	holds UI state and connects domain → components	yes

Design decisions (justified)

1. **Domain logic as pure functions without DOM/React.** All rules (`validateInterval`, `checkAvailability`, `filterRooms`, `availableRooms`) take plain data and return plain data. *Why:* they can be tested in milliseconds in Node without a browser (45 of the 49 tests are pure-function tests), and the UI cannot re-

implement a rule by accident. This removed the 4 copies of time/duration logic that lived in `App.tsx` and `Timeline.tsx`.

2. **One validation function shared by the decision and the hints.** `validateInterval()` is the only place that knows the AC1/AC2 rules; the duration hint uses `durationOf()` / `exceedsMaxDuration()` from the same file. *Why:* before, the red "exceeds 120 minutes" hint and the real decision were two separate comparisons that could drift apart.
3. **Immutable data: deep-frozen catalog + `readonly` types (AC5).** *Why:* the compiler now rejects `room.bookings.push(...)`, and at run time a mutation throws `TypeError` (tested). A shallow `Object.freeze` was not enough (see AI suggestion #1).
4. **Derived result instead of stored result (F5).** The result is computed from `(checked, selectedRoom, start, end)` with `useMemo`, not stored with `setResult`. *Why:* it cannot get out of sync with the selected room. Switching rooms automatically re-checks against that room's bookings, and a filtered-out room cannot keep showing an old answer.
5. **Injectable data for tests.** `App` accepts optional `rooms` and `currentTime` props (defaults: the catalog and `FIXED_NOW`). *Why:* UI tests can use deterministic data without module mocking.

Behaviour kept consistent

- All 30 original tests still pass. The only edits to them: group titles renamed to the brief's AC numbers, imports updated after the file move, and `equipment: []` added to the test fixture once the model gained that field.
- User-facing messages are identical (strings moved, not rewritten).
- One intentional change: the live clock was replaced by a fixed time (08:00), as the brief requires.

Task #04 — Automated testing

Command: `npx vitest run` (same as `npm test`). Full verbose output: `docs/evidence/01-final-tests.txt`.

```
✓ tests/filter.test.ts (11 tests)
✓ tests/regression.test.ts (34 tests)
✓ tests/app.ui.test.tsx (4 tests)
Test Files  3 passed (3)
     Tests  49 passed (49)
```

Area	File	What is checked (boundaries in bold)
AC1 valid time	`regression.test.ts`	start = end, start > end, outside 00:00–24:00, not after "now", empty (null) start/end, malformed labels
AC2 120-min max	`regression.test.ts`	**exactly 120 accepted**, **121 rejected**
AC3 blocked	`regression.test.ts`	blocked room always Unavailable
AC4 overlap	`regression.test.ts`	contained, leading, trailing, identical, enveloping; **touching 10:00–11:00 / 11:00–12:00 allowed**
AC5 immutable	`regression.test.ts`, `filter.test.ts`	catalog deep-equal after checks and filters; `push` into bookings throws
AC6 feedback	`regression.test.ts`, UI test	decision + reason text names the room / the clashing booking
Catalog	`filter.test.ts`	≥4 rooms, all required fields, unique ids

Area	File	What is checked (boundaries in bold)
F1 capacity	`filter.test.ts`	**capacity = attendees kept**, **attendees = capacity + 1 dropped**, empty = no limit, invalid input rejected
F2 equipment	`filter.test.ts`	room with only part of the equipment excluded; capacity + equipment combined
F3 blocked	`filter.test.ts`	blocked room matching every filter is not in `availableRooms`
F4 empty	`filter.test.ts`, UI test	empty list; empty-state text in the DOM; *Clear filters* restores the list
F5 room switch	`filter.test.ts`, UI test	rooms A and B give opposite results for the same interval; in the DOM, clicking another room changes the result
UI integration	`app.ui.test.tsx` (jsdom + Testing Library)	typing attendees / ticking equipment updates the rendered list; empty state; room-switch re-check; inline attendee error

Can the tests fail? Six rules were broken on purpose (mutation check, `docs/evidence/02-mutation-check.txt`). Every mutant made at least one test fail:

Mutant	Failing tests
overlap `<` → `<=` (touching = conflict)	4
capacity `>=` → `>`	3
equipment `every` → `some`	6+
duration `>` → `>=` 120	1
deep freeze → shallow freeze	1
room switch resets instead of re-checking	1

Other checks: `npx tsc --noEmit -p tsconfig.app.json` → 0 errors; `npx eslint .` → 0 problems.

Task #05 — Quality evaluation (before → after)

Measure	Before (`v0-baseline`)	After (`HEAD`)	How it was obtained
Automated tests passed / total	**30 / 30** (1 file)	**49 / 49** (3 files)	`npx vitest run` → `docs/evidence/00-baseline-tests.txt`, `01-final-tests.txt`
Tests for AC5 / F1–F5 / UI behaviour	0 / 0 / 0	3 / 9 (+1 catalog) / 4	counted `it(` blocks per `describe` in `tests/`
Mutants killed	not measured	6 / 6	`docs/evidence/02-mutation-check.txt`
Duplicated logic/rendering locations	**8**	**0**	same `git grep -nE` pattern on both versions (`docs/evidence/03-metrics.txt`); the remaining matches are the single canonical definitions in `domain/time.ts` and `SectionHeading.tsx`
Lint problems	1 error, 1 warning	0	`npx eslint .`
Type errors	0	0	`npx tsc --noEmit -p tsconfig.app.json`

Measure	Before (`v0-baseline`)	After (`HEAD`)	How it was obtained
Code structure	3 `lib/` files mixing helpers, rules and data; logic also in 2 components	7 domain modules (incl. types) + 1 data module + 6 presentation components + `App` wiring	file tree; table in Task #03
Rooms / filters	3 rooms, no filters	5 rooms, capacity + equipment filters	`src/data/rooms.ts`
Shared data mutable?	yes (`ROOMS: Room[]`)	no (deep-frozen, `readonly`)	AC5 tests
Runtime dependencies	4 (incl. unused Supabase)	3	`package.json`
Size of change	—	+879 / −318 lines in `src` + `tests`	`git diff --stat v0-baseline..HEAD -- src tests`

Remaining limitations

1. **`App.tsx` is still the largest file (224 lines vs 222).** It no longer contains rules, but it still owns all UI state. The next step would be a `useRoomSearch()` hook so state transitions can be unit-tested on their own.
2. **Single day, check-only.** Bookings are fixed fictional data for "today"; nothing can be created or saved (by design of the brief, but it limits realism).
3. **`availableRooms()` is O(n^2)** because each check looks the room up again in the catalog. Irrelevant for 5 rooms, but a real catalog would need an index.
4. **UI coverage is behavioural, not visual.** The jsdom tests do not check layout; responsiveness was verified only by one 390-px screenshot, not by an automated test.

Task #06 — AI reflection

Context I gave the assistant. I worked with Claude Code (Claude Opus 5.5) inside my repository, so it could read the code itself. I gave it the assignment PDF, my own step-by-step plan, the acceptance criteria AC1–AC6, the new rules F1–F5 and the constraints: no login, database or live API, fictional data, fixed current time, and small steps instead of a rewrite.

Useful and incorrect suggestions. The most useful outputs were the risk list with exact file and line references, the idea of a single `validateInterval()` used by both the decision and the UI hint, and the mutation check, which showed that the tests really fail when a rule breaks. Several suggestions were wrong or weak. The plan assumed vanilla JavaScript and no existing tests. It proposed a shallow `Object.freeze`, which would not have protected nested bookings. One draft silently treated invalid attendee input as "no filter". Another used type casts and an `eslint-disable` to hide problems instead of fixing them, and the first metrics table miscounted tests and components.

What I verified personally. I re-ran `npx vitest run`, `tsc` and `eslint` and checked that the outputs matched the saved evidence. I opened the referenced lines in `v0-baseline` and went through the filtering scenarios in the browser. I also compared the free-room count with the booking data by hand.

How tests influenced decisions. The 30 existing tests were the safety net that allowed moving files and merging validation without fear. The AC5 test forced deep freezing instead of shallow freezing. The UI test for F5 led to computing the result from the selected room instead of storing it.

What I would improve next. I would move the UI state into a `useRoomSearch()` hook, add an automated responsive-layout check, and index rooms by id.

AI-use record (Assignment #04)

AI assistant used: Claude Code (desktop app, Code tab), model **Claude Opus 5.5** (`claude-opus-5-5`), Anthropic.
Mode of use: the assistant read the repository, proposed plans, edited files and ran commands in my local checkout; I gave the requirements, reviewed every diff and the test output, and decided what to keep.

Each entry: date · tool · prompt summary · output used · problems found · changes made · how verified.

---

1. 2026-11-08 · Claude Code · onboarding / baseline check

- **Prompt summary:** gave the assignment PDF and my step-by-step plan; asked whether my current repo (Practice #08 result) can be the starting version.
- **Output used:** inventory of the repo (React + TS + Vite + Vitest, 30 existing tests, `src/lib/availability.ts` already pure), list of gaps vs. the brief (3 rooms instead of ≥4, no `equipment`, real clock instead of a fixed current time, unused `@supabase/supabase-js`, AC numbering in tests not matching the brief, no UI test).
- **Problems found:** first answer had not actually read the PDF (no PDF tooling on the machine) and said so; after installing Python it extracted the text and pointed out the deadline is **today (2026-11-08 23:59)**, not tomorrow as my plan assumed.
- **Changes made:** compressed the plan into one day.
- **Verified:** ran `npx vitest run` myself → `Tests 30 passed (30)`; `npx tsc --noEmit -p tsconfig.app.json` → no errors; `npx eslint .` → 1 error, 1 warning. Saved to `docs/evidence/00-baseline-tests.txt`.
- **Baseline:** tag `v0-baseline` → commit `9badd8c`.

2. 2026-11-08 · Claude Code · Task #01 codebase analysis

- **Prompt summary:** "Follow my plan": repository map + AC1–AC6 + F1–F5 + constraints (no login/DB/API, fictional data, fixed time, small steps).
- **Output used:** 7 risks with file:line references (`docs/REPORT.md` Task #01), an 8-item duplicated-logic inventory, and a prioritized plan P0–P5.
- **Problems found:** the plan I had pasted (AI-generated earlier) assumed vanilla JS (`main.js`, `ui/render.js`), assumed there were no tests, and proposed `Object.freeze` for AC5. All three were adapted (see REPORT, "AI suggestions rejected or modified").
- **Verified:** opened every referenced line in `git show v0-baseline:<file>` and confirmed it; the duplicate inventory was re-checked with `git grep -nE` (`docs/evidence/03-metrics.txt`).

3. 2026-11-08 · Claude Code · P0 baseline fixes

- **Output used:** removed unused `@supabase/supabase-js`; replaced the live `new Date()` clock with `FIXED_NOW = 08:00`.
- **Verified:** `grep -c supabase package.json package-lock.json` → 0; tests 30/30 after each commit.

4. 2026-11-08 · Claude Code · Task #03 refactoring (4 commits)

- **Output used:** move to `data/` + `domain/` + `tests/`; one `validateInterval()`; `deepFreeze` + `readonly` types; `SectionHeading` component.
- **Problems found:** the assistant's first version of the refactor inside `checkAvailability` used `start as number` casts in four places after validation. Rejected as hiding the null case from the compiler; changed so `validateInterval()` returns the narrowed `{ start, end }`. One automated text replacement (regex over JSX headings) silently matched nothing ("replaced 0"); it was caught because `grep -n "<h2"` still showed three headings, and was redone by line numbers.
- **Verified:** after each step `npx tsc`, `npx vitest run`, `npx eslint .`; reviewed each `git diff` before committing.

5. 2026-11-08 · Claude Code · Task #02 feature

- **Output used:** 5-room catalog with equipment; `domain/filter.ts`; `RoomFilters` component; empty state; derived result for F5.
- **Problems found / modified:** first draft of `parseAttendees` turned invalid input (e.g. `0`) silently into "no filter". Changed to return an error that is shown inline, because silent acceptance hides a mistake from the user (AC6 clear feedback). The first draft of `App` also used an `eslint-disable` to make a `useMemo` dependency list pass; replaced with a plain derived value.
- **Verified:** manually in the in-app browser: checked Library A 09:00–10:00 → Unavailable, switched to Engineering Lab 3 → Available, Design Studio 2 → blocked; "2 of 5 matching rooms are free 09:00–10:00" matched the data by hand.

6. 2026-11-08 · Claude Code · Task #04 tests

- **Output used:** `tests/filter.test.ts` (11), `tests/app.ui.test.tsx` (4, jsdom), extra AC1/AC5 tests in `regression.test.ts`.
- **Problems found:** all new tests passed on the first run, which by itself proves nothing. So six rules were broken on purpose (mutation check). All six made tests fail (`docs/evidence/02-mutation-check.txt`).
- **Verified:** `npx vitest run` → 49 passed (49).

7. 2026-11-08 · Claude Code · Task #05 metrics, screenshots, README

- **Output used:** before/after table, Playwright script driving Edge for 9 screenshots, README.
- **Problems found:** the first metrics table over-counted (said 11 F1–F5 tests, the real number is 9 + 1 catalog test; said 7 components, there are 6). Corrected after recounting the `it(` blocks and files.
- **Verified:** every number in the table comes from a command saved in `docs/evidence/`.

Decisions I kept for myself

- Keep blocked rooms visible in the filtered list (with a *Blocked* badge) instead of hiding them: the user should see *why* a room they expect is not available, and F3 only forbids reporting it as available.
- Keep the original 30 tests instead of rewriting them.
- No new `SKILL.md` (the brief forbids it).

Appendix A — Commit history (one change per commit)

Command: `git log --format="%h %ad %s" v0-baseline..HEAD`

```
ff2d607 2026-11-08 15:48 docs: README, full report (Tasks 01-06) and AI-use log
27cfce9 2026-11-08 15:45 docs: scenario screenshots and before/after metrics evidence
a429ad9 2026-11-08 15:44 test: F1-F5 filter tests, jsdom UI integration test, and test evidence
20da6de 2026-11-08 15:42 feat: filter UI, empty state and result refresh on room switch (F1-F5)
d9a84f4 2026-11-08 15:39 feat: capacity and equipment filtering rules in domain/filter (F1-F3)
794d0c6 2026-11-08 15:39 feat: room catalog of 5 fictional rooms with equipment lists
686a408 2026-11-08 15:38 refactor: extract SectionHeading and remove duplicate time helper from TimeField
23276ce 2026-11-08 15:37 refactor: deep-freeze room data and make domain types readonly (AC5)
0d086a3 2026-11-08 15:37 refactor: one validateInterval/durationOf rule in domain/time, used by domain
and UI
6ffe4be 2026-11-08 15:35 refactor: split code into data/, domain/ (time, availability) and tests/
4ab296d 2026-11-08 15:35 test: align regression test groups with AC1-AC6 and add AC5 immutability check
0e3c3fe 2026-11-08 15:34 fix: use a fixed current time (08:00) instead of the live clock
2727dc9 2026-11-08 15:34 chore: remove unused @supabase/supabase-js (no database allowed)
377114b 2026-11-08 15:33 docs: start AI-use log and record baseline test run
```

Appendix B — Final test run

Command: `npx vitest run --reporter=verbose`

```
 RUN  v2.1.9 C:/Users/tempadmin/Desktop/aibar/swe/practise 9/project

 ✓ tests/filter.test.ts > Catalog > has at least four rooms, each with id, name, capacity, equipment,
blocked status and bookings
 ✓ tests/filter.test.ts > F1 — Capacity filter (capacity >= attendees) > keeps a room whose capacity is
exactly the number of attendees
 ✓ tests/filter.test.ts > F1 — Capacity filter (capacity >= attendees) > drops a room whose capacity is
one less than the attendees
 ✓ tests/filter.test.ts > F1 — Capacity filter (capacity >= attendees) > applies no capacity limit when
attendees is empty
 ✓ tests/filter.test.ts > F1 — Capacity filter (capacity >= attendees) > rejects zero, negative and
fractional attendee input with a message
 ✓ tests/filter.test.ts > F2 — Equipment filter (ALL selected items required) > excludes a room that has
only part of the selected equipment
 ✓ tests/filter.test.ts > F2 — Equipment filter (ALL selected items required) > combines capacity and
equipment (both must hold)
 ✓ tests/filter.test.ts > F3 — A blocked room is never reported as available > omits a blocked room from
available rooms even when it matches every filter
 ✓ tests/filter.test.ts > F4 — Empty result when nothing matches > returns an empty list when no room
satisfies the requirements
 ✓ tests/filter.test.ts > F5 — Availability uses the chosen room bookings > gives different results for
room A and room B for the same interval
 ✓ tests/filter.test.ts > AC5 — Filtering does not modify the catalog > leaves rooms deep-equal and in
the same order after filtering
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > intervalsOverlap (pure
boundary helper) > T01: rejects a fully-contained overlap
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > intervalsOverlap (pure
boundary helper) > T02: rejects a partial overlap on the leading edge
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > intervalsOverlap (pure
boundary helper) > T03: rejects a partial overlap on the trailing edge
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > intervalsOverlap (pure
```

```
boundary helper) > T04: allows a booking that ends exactly when the existing one starts (touching)
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > intervalsOverlap (pure
boundary helper) > T05: allows a booking that starts exactly when the existing one ends (touching)
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > intervalsOverlap (pure
boundary helper) > T06: allows a booking entirely before the existing one
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > intervalsOverlap (pure
boundary helper) > T07: allows a booking entirely after the existing one
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > intervalsOverlap (pure
boundary helper) > T08: rejects an identical-time overlap (duplicate constraint)
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > checkAvailability
boundary reflection > T09: marks a touching booking as Available
 ✓ tests/regression.test.ts > AC4 — Overlap rules (touching intervals allowed) > checkAvailability
boundary reflection > T10: marks an overlapping booking as Unavailable and names the clash
 ✓ tests/regression.test.ts > AC3 / AC6 — Blocked room and clear status feedback > T11: returns a clear
Available status with the room name and interval
 ✓ tests/regression.test.ts > AC3 / AC6 — Blocked room and clear status feedback > T12: returns a clear
Booked (unavailable) status when the room is blocked
 ✓ tests/regression.test.ts > AC3 / AC6 — Blocked room and clear status feedback > T13: returns a clear
Booked (unavailable) status when overlapping a real booking
 ✓ tests/regression.test.ts > AC3 / AC6 — Blocked room and clear status feedback > reflects Available for
the sample Library Study Room A on a free slot
 ✓ tests/regression.test.ts > AC3 / AC6 — Blocked room and clear status feedback > reflects Unavailable
for the blocked Design Studio 2 regardless of time
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > T14: rejects
when start equals end (zero-length interval)
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects when
start is after end (inverted range)
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects an end
time beyond 24:00 (MINUTES_PER_DAY boundary)
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects a
start time before 00:00 (negative boundary)
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects when
start is not strictly after the current time
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects when
the candidate starts in the past (before current time)
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects a
duration exceeding the 120-minute limit
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > accepts a
duration exactly equal to the 120-minute limit
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects when
the room id does not exist (missing field / bad reference)
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects a
duplicate booking that exactly matches an existing one
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects a
booking that fully envelops an existing one
 ✓ tests/regression.test.ts > AC1 / AC2 — Valid time and 120-minute maximum (edge cases) > rejects a
booking fully contained inside an existing one
 ✓ tests/regression.test.ts > time conversion utilities > converts minutes to HH:MM labels correctly
 ✓ tests/regression.test.ts > time conversion utilities > converts valid HH:MM labels to minutes
 ✓ tests/regression.test.ts > time conversion utilities > rejects malformed or out-of-range time labels
 ✓ tests/regression.test.ts > AC5 — Immutable booking data > leaves the shared ROOMS catalog deep-equal
after many checks
 ✓ tests/regression.test.ts > AC1 — Empty or malformed time input is rejected by the domain > rejects an
empty start time (null) with a clear message
 ✓ tests/regression.test.ts > AC1 — Empty or malformed time input is rejected by the domain > rejects an
empty end time (null)
 ✓ tests/regression.test.ts > AC5 — Room data is deep-frozen > throws when code tries to push into a room
booking list (nested freeze, not shallow)
 ✓ tests/app.ui.test.tsx > UI integration (React + jsdom) > shows all rooms, then updates the DOM list
when filters change (F1, F2)
 ✓ tests/app.ui.test.tsx > UI integration (React + jsdom) > shows a clear empty state and can clear the
filters (F4)
 ✓ tests/app.ui.test.tsx > UI integration (React + jsdom) > re-checks the selected room when the user
switches rooms (F5, F3)
 ✓ tests/app.ui.test.tsx > UI integration (React + jsdom) > shows an inline message for invalid attendee
```

```
input

 Test Files  3 passed (3)
      Tests  49 passed (49)
   Start at  15:44:31
   Duration  3.60s (transform 352ms, setup 0ms, collect 1.06s, tests 535ms, environment 1.10s, prepare
2.19s)
```

Appendix C — Baseline test run

```
 RUN  v2.1.9 C:/Users/tempadmin/Desktop/aibar/swe/practise 9/project

 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > intervalsOverlap (pure boundary helper) > T01: rejects a fully-contained overlap
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > intervalsOverlap (pure boundary helper) > T02: rejects a partial overlap on the leading edge
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > intervalsOverlap (pure boundary helper) > T03: rejects a partial overlap on the trailing edge
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > intervalsOverlap (pure boundary helper) > T04: allows a booking that ends exactly when the existing one starts (touching)
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > intervalsOverlap (pure boundary helper) > T05: allows a booking that starts exactly when the existing one ends (touching)
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > intervalsOverlap (pure boundary helper) > T06: allows a booking entirely before the existing one
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > intervalsOverlap (pure boundary helper) > T07: allows a booking entirely after the existing one
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > intervalsOverlap (pure boundary helper) > T08: rejects an identical-time overlap (duplicate constraint)
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > checkAvailability boundary reflection > T09: marks a touching booking as Available
 ✓ src/lib/roomAvailability.test.ts > AC1 — Overlap & boundary logic > checkAvailability boundary reflection > T10: marks an overlapping booking as Unavailable and names the clash
 ✓ src/lib/roomAvailability.test.ts > AC2 — State & status validation > T11: returns a clear Available status with the room name and interval
 ✓ src/lib/roomAvailability.test.ts > AC2 — State & status validation > T12: returns a clear Booked (unavailable) status when the room is blocked
 ✓ src/lib/roomAvailability.test.ts > AC2 — State & status validation > T13: returns a clear Booked (unavailable) status when overlapping a real booking
 ✓ src/lib/roomAvailability.test.ts > AC2 — State & status validation > reflects Available for the sample Library Study Room A on a free slot
 ✓ src/lib/roomAvailability.test.ts > AC2 — State & status validation > reflects Unavailable for the blocked Design Studio 2 regardless of time
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > T14: rejects when start equals end (zero-length interval)
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects when start is after end (inverted range)
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects an end time beyond 24:00 (MINUTES_PER_DAY boundary)
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects a start time before 00:00 (negative boundary)
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects when start is not strictly after the current time
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects when the candidate starts in the past (before current time)
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects a duration exceeding the 120-minute limit
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > accepts a duration exactly equal to the 120-minute limit
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects when the room id does not exist (missing field / bad reference)
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects a duplicate booking that exactly matches an existing one
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects a booking that fully envelops an existing one
 ✓ src/lib/roomAvailability.test.ts > AC3 — Edge cases > rejects a booking fully contained inside an existing one
 ✓ src/lib/roomAvailability.test.ts > time conversion utilities > converts minutes to HH:MM labels correctly
 ✓ src/lib/roomAvailability.test.ts > time conversion utilities > converts valid HH:MM labels to minutes
 ✓ src/lib/roomAvailability.test.ts > time conversion utilities > rejects malformed or out-of-range time
```

```
labels

 Test Files  1 passed (1)
      Tests  30 passed (30)
   Start at  15:33:01
   Duration  1.40s (transform 105ms, setup 0ms, collect 120ms, tests 11ms, environment 0ms, prepare
184ms)
```

Appendix D — Mutation check and metrics commands

```
Mutation check — 2026-11-08, run on the final code.
Each rule was broken on purpose with sed, `npx vitest run` was run, then the file was restored with `git
checkout`.
A mutant is "killed" when at least one test fails.

M1 overlap uses <= / >= (touching = conflict)         -> 4 failed | 45 passed (49)  KILLED (T04, T05,
T09, F3)
M2 capacity uses > instead of >=                       -> 3 failed | 46 passed (49)  KILLED (F1 exact
capacity, F2 combined, UI F1/F2)
M3 equipment uses some() instead of every()            -> 6+ failed                  KILLED (F1, F2, UI
F1/F2/F4/F5)
M4 max duration uses >= 120 instead of > 120           -> 1 failed | 48 passed (49)  KILLED (exactly 120
accepted)
M5 deepFreeze replaced by shallow freeze               -> 1 failed | 48 passed (49)  KILLED (AC5 deep-
frozen)
M6 room switch resets result instead of re-checking    -> 1 failed | 48 passed (49)  KILLED (UI F5)

Result: 6 / 6 mutants killed. All files restored; `git status` clean apart from the new tests.
```

```
$ git grep -nE "=== null \|\| endMin|endMin > startMin|> MAX_BOOKING_MINUTES|candidateStart >=
0|padStart|minutesToTimeInput|uppercase tracking-wide" v0-baseline -- src ":!*.test.ts"
v0-baseline:src/App.tsx:26:  const hh = d.getHours().toString().padStart(2, '0');
v0-baseline:src/App.tsx:27:  const mm = d.getMinutes().toString().padStart(2, '0');
v0-baseline:src/App.tsx:28:  const ss = d.getSeconds().toString().padStart(2, '0');
v0-baseline:src/App.tsx:58:    if (startMin === null || endMin === null) {
v0-baseline:src/App.tsx:82:    startMin !== null && endMin !== null && endMin > startMin
v0-baseline:src/App.tsx:86:  const durationOver = duration !== null && duration > MAX_BOOKING_MINUTES;
v0-baseline:src/App.tsx:110:          <h2 className="mb-3 flex items-center gap-2 text-sm font-semibold
uppercase tracking-wide text-slate-500">
v0-baseline:src/App.tsx:128:          <h2 className="mb-4 flex items-center gap-2 text-sm font-semibold
uppercase tracking-wide text-slate-500">
v0-baseline:src/App.tsx:207:          <h2 className="mb-3 flex items-center gap-2 text-sm font-semibold
uppercase tracking-wide text-slate-500">
v0-baseline:src/components/TimeField.tsx:21:export function minutesToTimeInput(min: number): string {
v0-baseline:src/components/Timeline.tsx:66:          candidateStart >= 0 && candidateEnd <=
MINUTES_PER_DAY && candidateStart < candidateEnd && (
v0-baseline:src/lib/availability.ts:9:  const hh = h.toString().padStart(2, '0');
v0-baseline:src/lib/availability.ts:10:  const mm = m.toString().padStart(2, '0');
v0-baseline:src/lib/availability.ts:70:  if (end - start > MAX_BOOKING_MINUTES) {

$ git grep -nE "<same pattern>" HEAD -- src
HEAD:src/components/SectionHeading.tsx:11:      className={`${className} flex items-center gap-2 text-sm
font-semibold uppercase tracking-wide text-slate-500`}
HEAD:src/domain/time.ts:9:  const hh = h.toString().padStart(2, '0');
HEAD:src/domain/time.ts:10:  const mm = m.toString().padStart(2, '0');
HEAD:src/domain/time.ts:46:  return duration !== null && duration > MAX_BOOKING_MINUTES;

$ git diff --stat v0-baseline..HEAD -- src tests
 src/App.tsx                                        | 176 ++++++++++++----------
```

```
 src/components/ResultPanel.tsx                     |   2 +-
 src/components/RoomFilters.tsx                     |  92 ++++++++++
 src/components/RoomSelector.tsx                    | 143 ++++++++++++------
 src/components/SectionHeading.tsx                  |  19 +++
 src/components/TimeField.tsx                       |   6 -
 src/components/Timeline.tsx                        |   8 +-
 src/data/rooms.ts                                  |  68 ++++++++
 src/domain/availability.ts                         |  55 +++++++
 src/domain/clock.ts                                |   6 +
 src/domain/filter.ts                               |  75 +++++++++
 src/domain/immutable.ts                            |  15 ++
 src/domain/time.ts                                 |  79 +++++++++
 src/domain/types.ts                                |  27 ++++
 src/lib/availability.ts                            |  98 ------------
 src/lib/data.ts                                    |  37 -----
 src/lib/types.ts                                   |  21 ---
 tests/app.ui.test.tsx                              |  62 ++++++++
 tests/filter.test.ts                               | 140 +++++++++++++++++
 .../regression.test.ts                             |  68 +++++++--
 20 files changed, 879 insertions(+), 318 deletions(-)

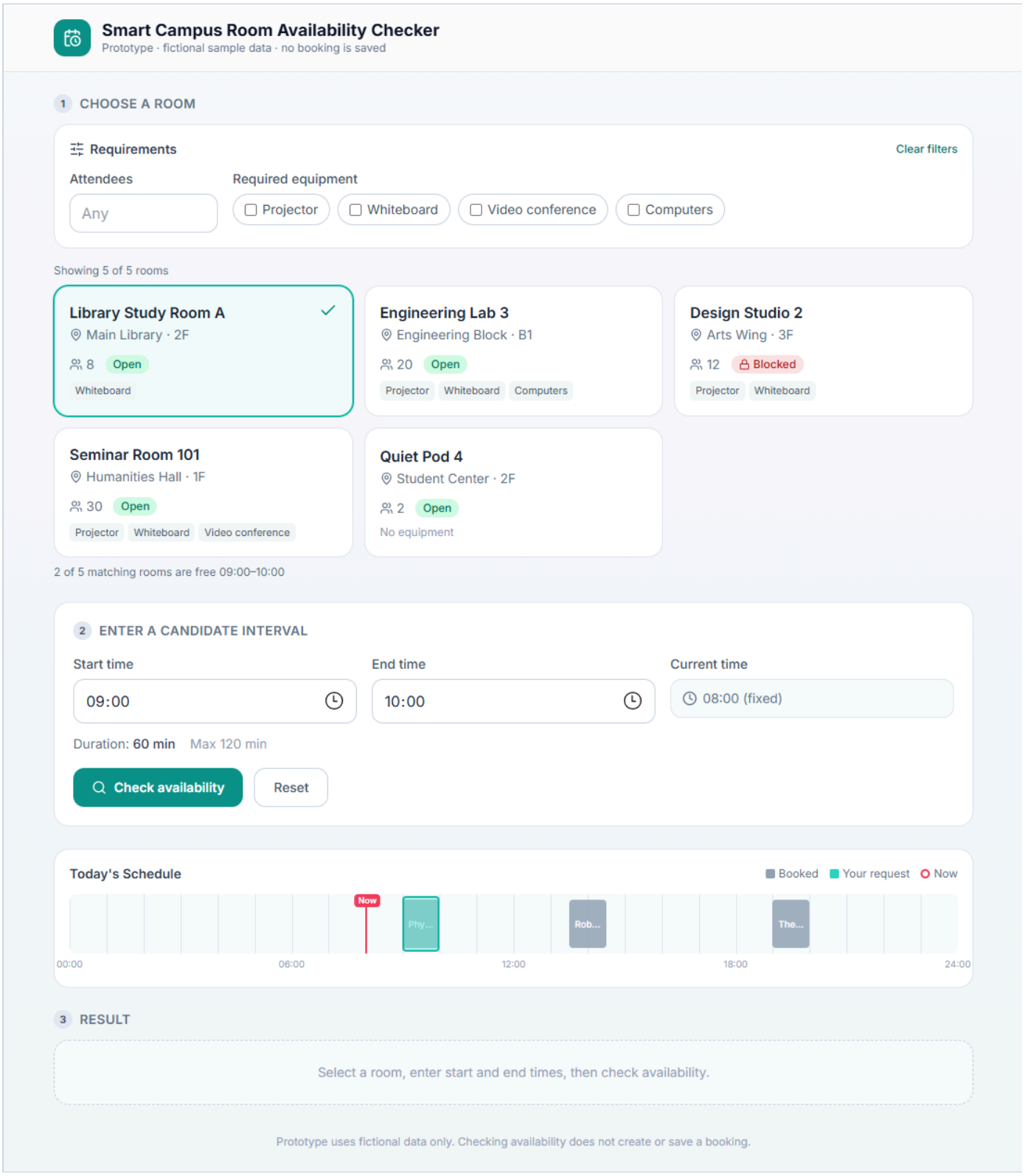
$ wc -l (baseline App.tsx vs final App.tsx)
222
224

$ npx eslint . (final)
0 problems

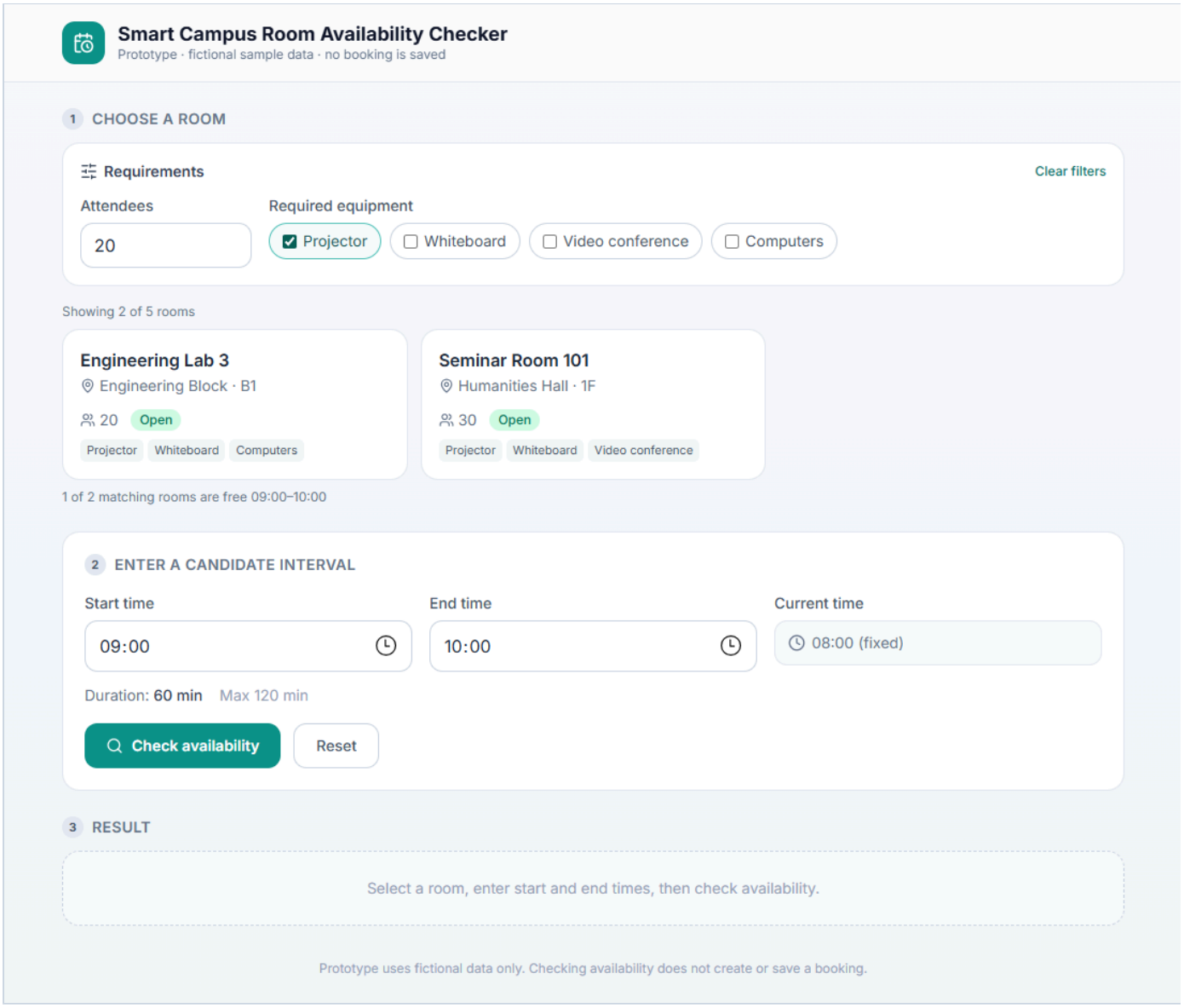
$ npx tsc --noEmit -p tsconfig.app.json (final)
0 errors

$ git log --oneline v0-baseline..HEAD
a429ad9 test: F1-F5 filter tests, jsdom UI integration test, and test evidence
20da6de feat: filter UI, empty state and result refresh on room switch (F1-F5)
d9a84f4 feat: capacity and equipment filtering rules in domain/filter (F1-F3)
794d0c6 feat: room catalog of 5 fictional rooms with equipment lists
686a408 refactor: extract SectionHeading and remove duplicate time helper from TimeField
23276ce refactor: deep-freeze room data and make domain types readonly (AC5)
0d086a3 refactor: one validateInterval/durationOf rule in domain/time, used by domain and UI
6ffe4be refactor: split code into data/, domain/ (time, availability) and tests/
4ab296d test: align regression test groups with AC1-AC6 and add AC5 immutability check
0e3c3fe fix: use a fixed current time (08:00) instead of the live clock
2727dc9 chore: remove unused @supabase/supabase-js (no database allowed)
377114b docs: start AI-use log and record baseline test run
```

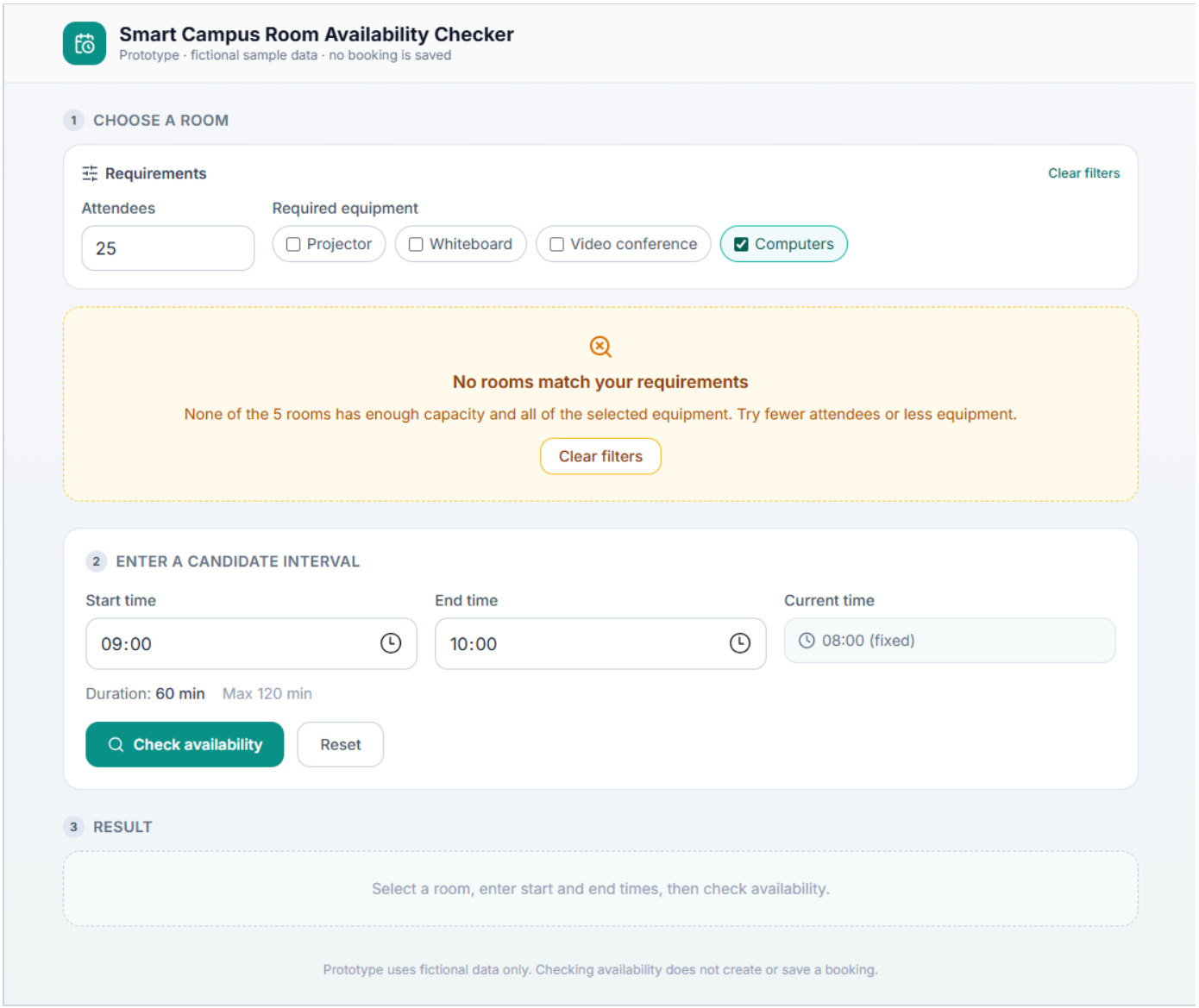
Appendix E — All screenshots



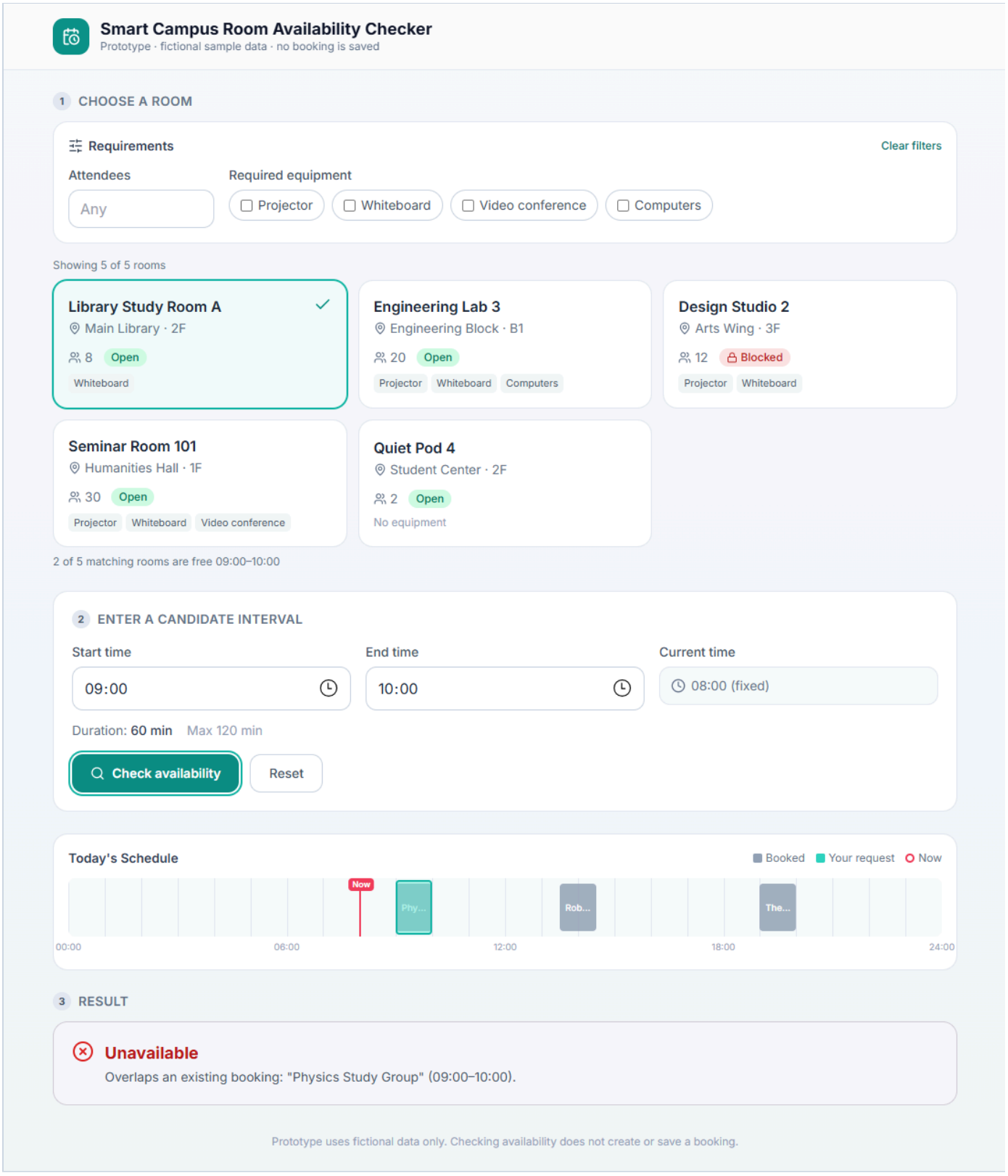
01-initial.png



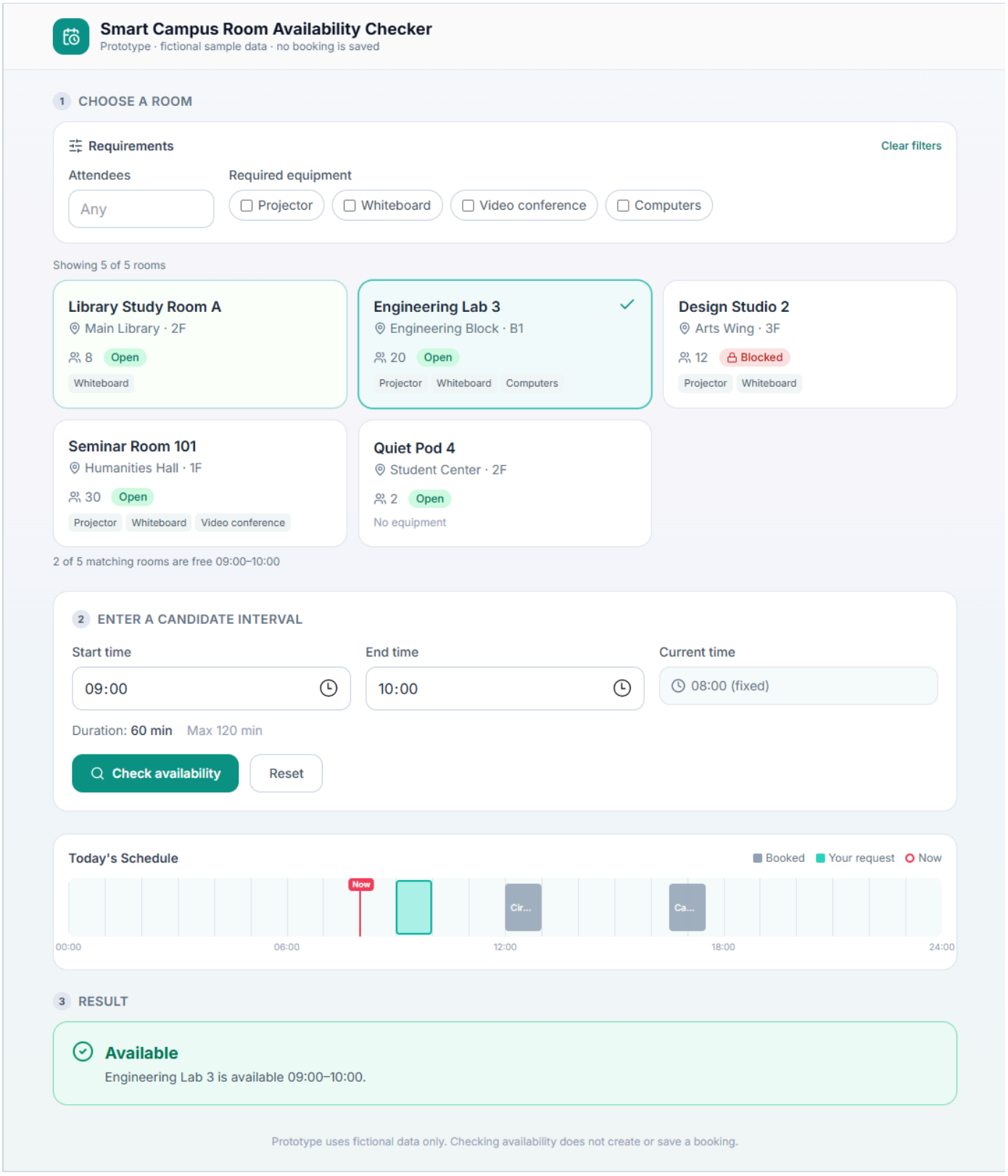
02-filter-success.png



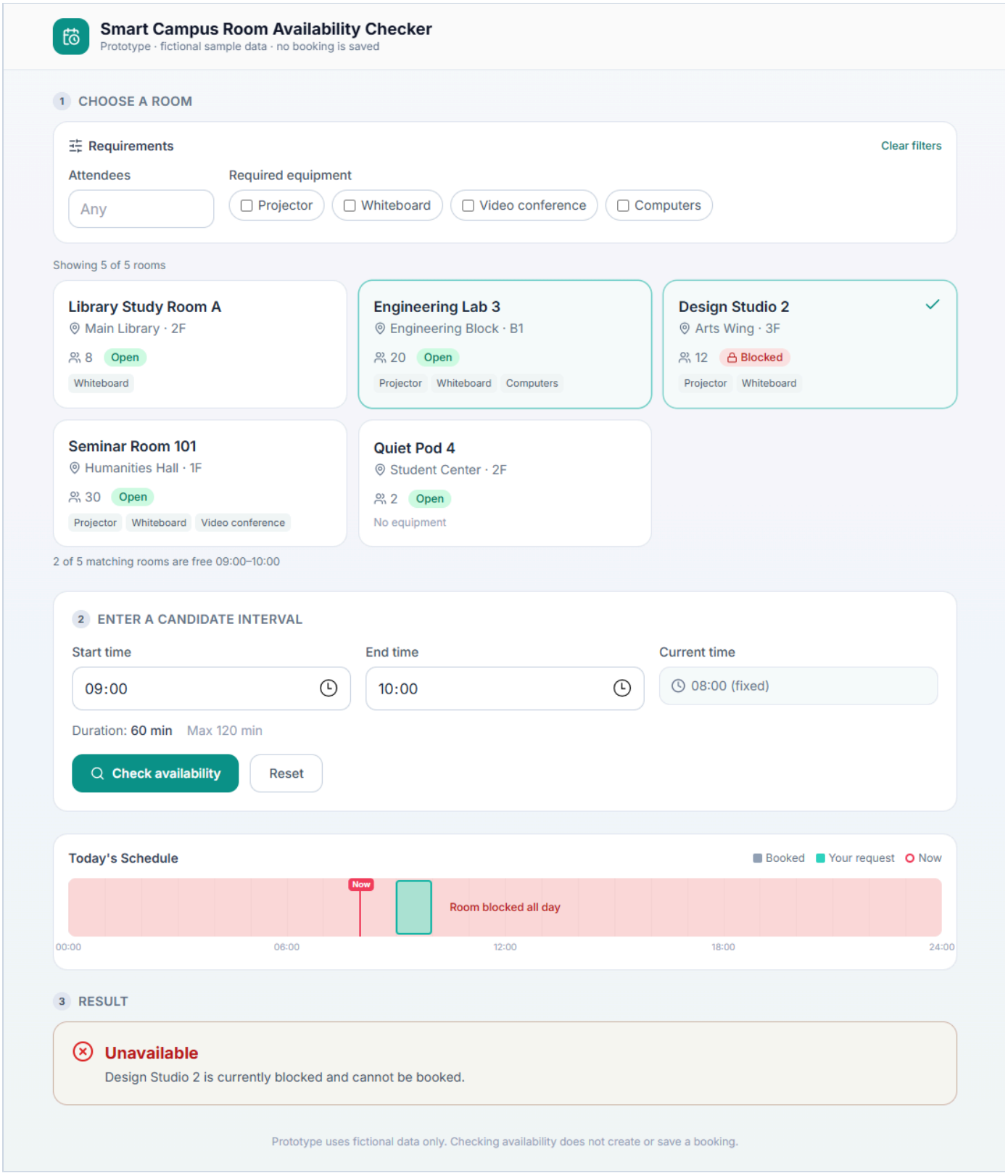
03-filter-empty.png



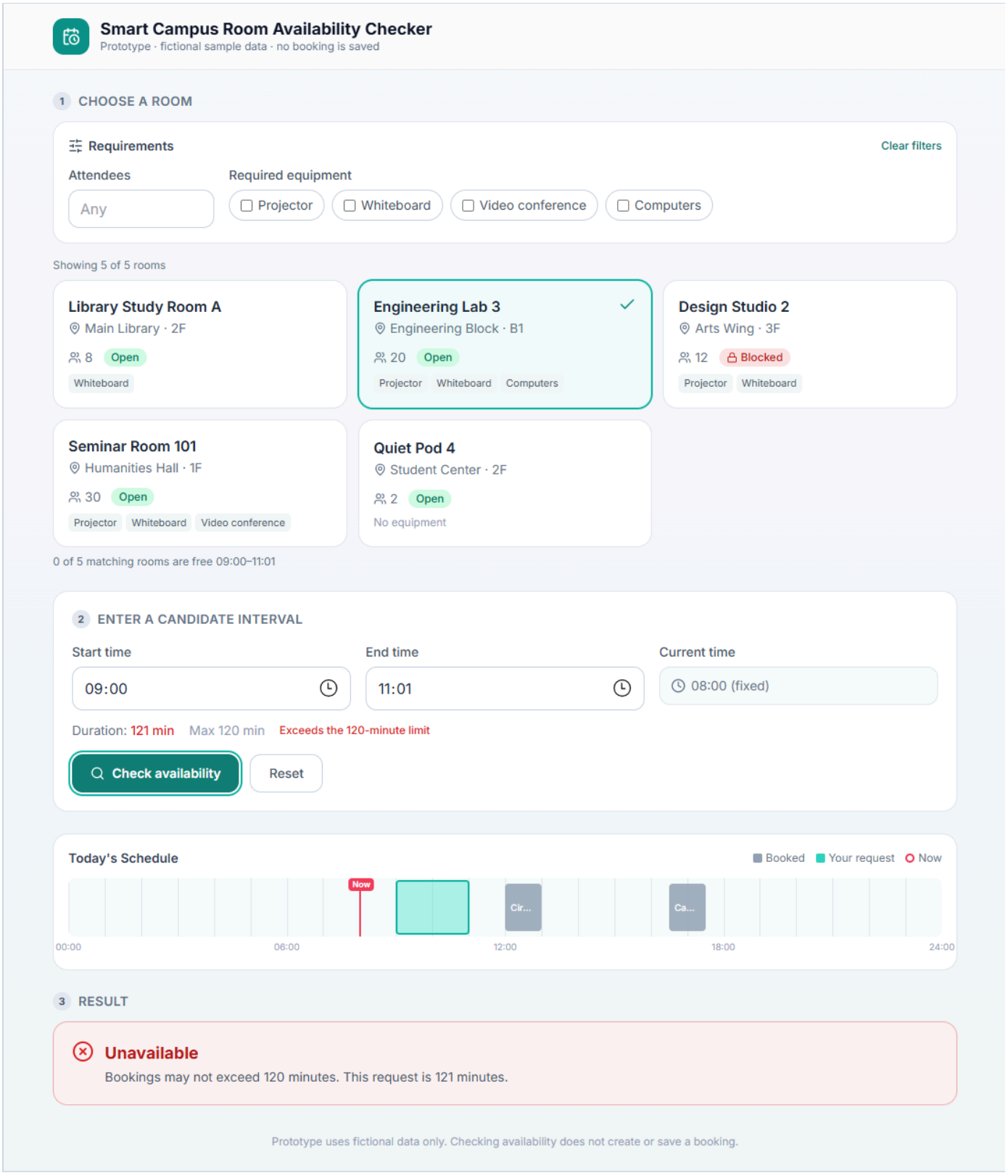
04-check-unavailable-overlap.png



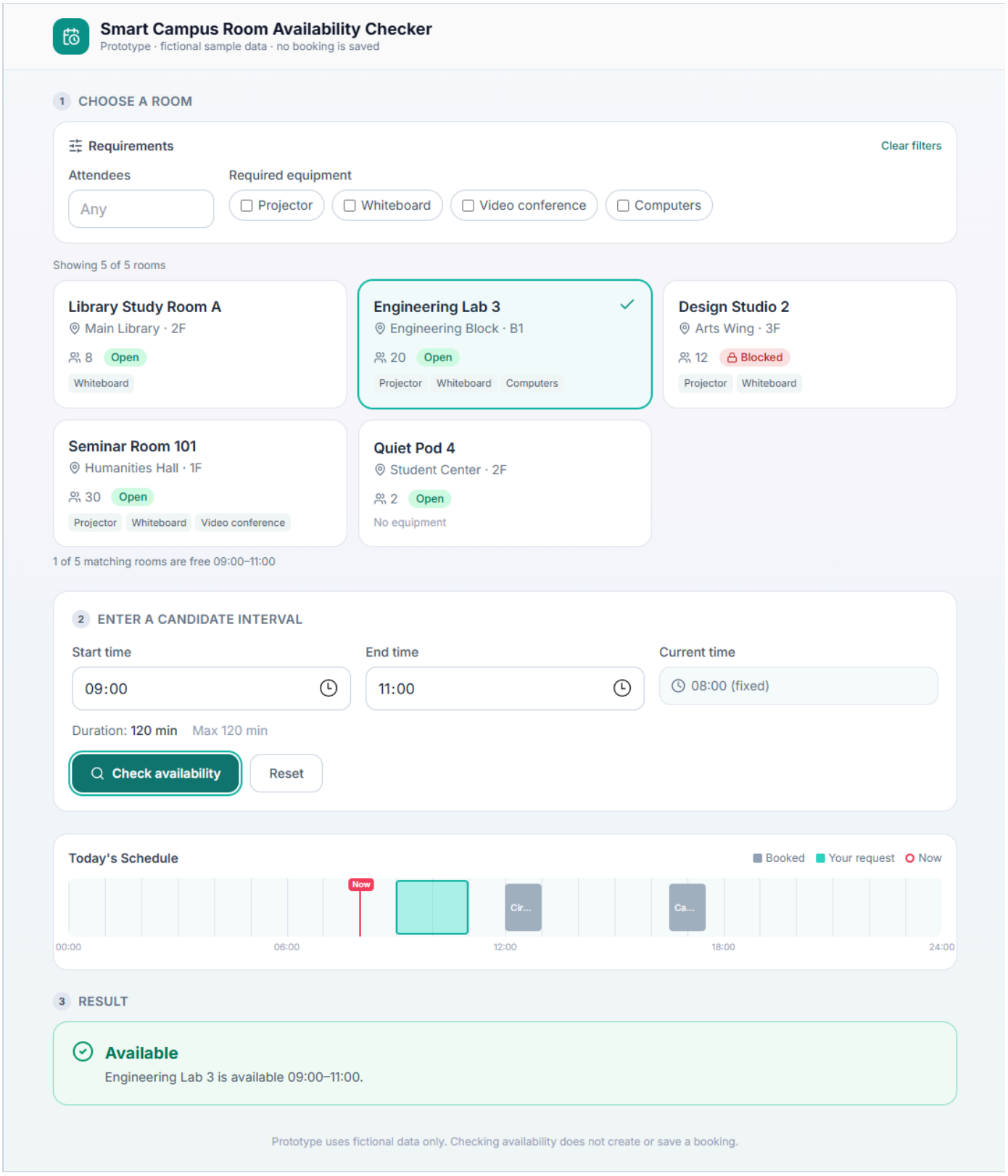
05-switch-room-available.png



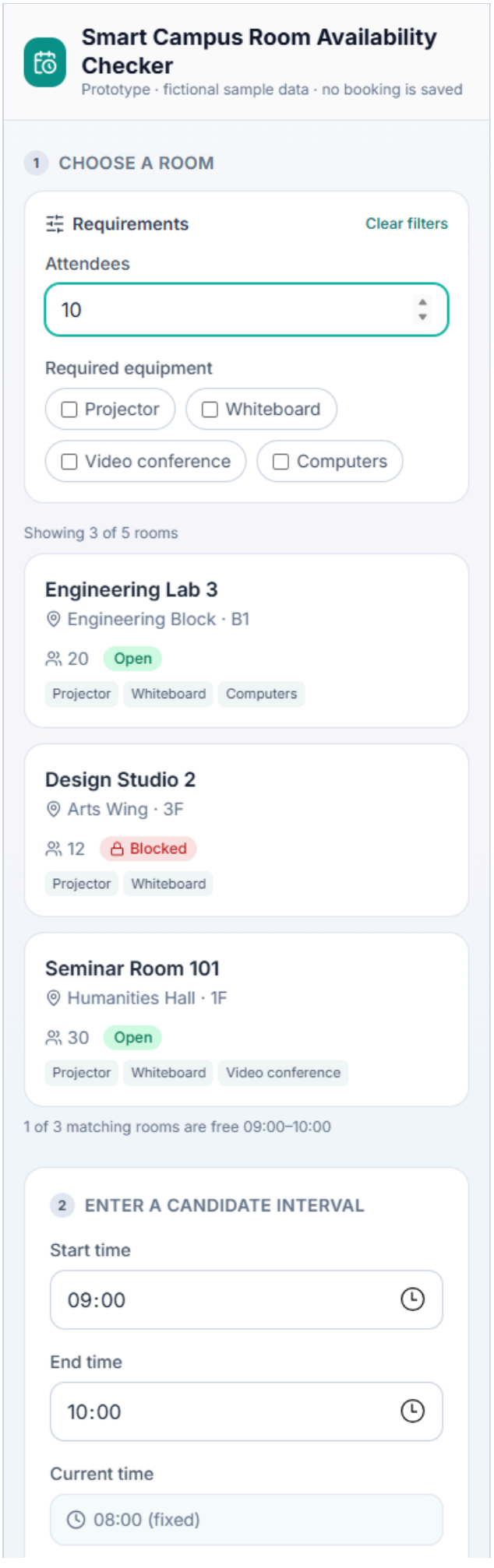
06-blocked-room.png

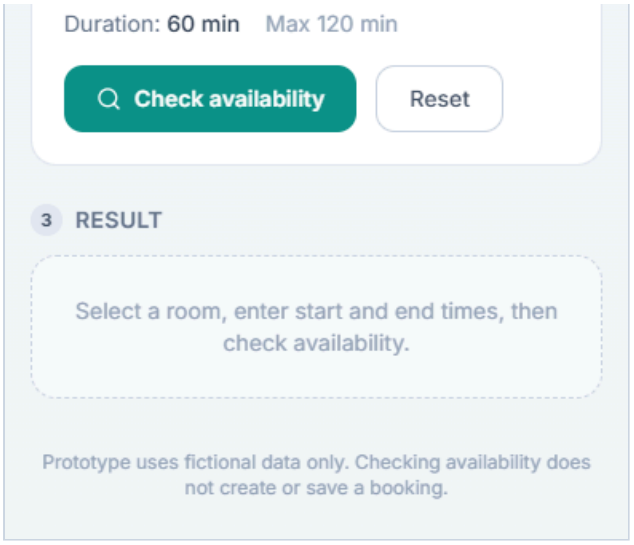


07-duration-121-rejected.png



08-duration-120-accepted.png


Smart Campus Room Availability Checker
Prototype · fictional sample data · no booking is saved
1 CHOOSE A ROOM
Requirements
Clear filters
Attendees
10
Required equipment
Projector
Whiteboard
Video conference
Computers
Showing 3 of 5 rooms
Engineering Lab 3
Engineering Block · B1
20
Open
Projector
Whiteboard
Computers
Design Studio 2
Arts Wing · 3F
12
Blocked
Projector
Whiteboard
Seminar Room 101
Humanities Hall · 1F
30
Open
Projector
Whiteboard
Video conference
1 of 3 matching rooms are free 09:00–10:00
2 ENTER A CANDIDATE INTERVAL
Start time
09:00
End time
10:00
Current time
08:00 (fixed)



09-mobile-390px.png